

API TESTING REPORT

Postman-Based Testing of the User Registration Endpoint (ASP.NET Core Identity)

Prepared with Postman

Name	Nour Albzoor
Date	Aug/9/2026
Company	Binx Company
Project	Task Tracker API — Week 4, Day 1
Feature Tested	User Registration Endpoint (ASP.NET Core Identity)
Endpoint	POST /api/Auth/register
Tool Used	Postman

1. Introduction

This report documents the functional testing of the User Registration endpoint, part of a REST API built with ASP.NET Core Identity. The testing was carried out using Postman, a widely used tool for building, sending, and validating API requests. The goal of this testing effort is to verify that the registration endpoint behaves as expected under both valid and invalid input conditions.

Before testing began, ASP.NET Core Identity was set up and the registration endpoint was implemented (see the hands-on lab summary below). The endpoint's required request body, validation rules, and expected response formats were then reviewed, and all requests were organized and executed in a structured Postman Collection under an Authentication folder.

2. Hands-On Lab: Set Up Identity & Implement Registration

Before testing could take place, ASP.NET Core Identity was configured and a registration endpoint was implemented in the Task Tracker API. The lab was completed in the following steps:

1	Add Identity NuGet packages and extend the DbContext Installed Microsoft.AspNetCore.Identity.EntityFrameworkCore (and Microsoft.EntityFrameworkCore.Tools) via NuGet, then updated the application's DbContext to inherit from IdentityDbContext instead of the base DbContext so the Identity schema is included in the model.
2	Create and apply the Identity migration

	Ran "dotnet ef migrations add AddIdentitySchema" to generate the migration for the Identity tables (AspNetUsers, AspNetRoles, AspNetUserRoles, etc.), then applied it with "dotnet ef database update" so the schema was created in the database.
3	Register Identity services in Program.cs Added builder.Services.AddIdentity<IdentityUser, IdentityRole>() (backed by AddEntityFrameworkStores and AddDefaultTokenProviders) in Program.cs so the DI container provides UserManager, RoleManager, and SignInManager, and configured the authentication middleware pipeline.
4	Implement the registration endpoint Built a POST /api/Auth/register endpoint that accepts a username, email, and password, calls UserManager.CreateAsync to create the user, and inspects the IdentityResult to return meaningful, field-level error messages (e.g. password complexity, duplicate email) when creation fails.
5	Test registration in Postman Verified the endpoint using two requests: a valid registration (strong password) that returns 200 OK, and a deliberately weak password that returns 400 Bad Request with Identity's validation error details. Full test results are documented in Sections 4-7 below.

3. Objective

The objective of this testing activity is to:

- Validate that the registration endpoint returns the correct HTTP status codes for valid and invalid requests.
- Confirm that a valid registration request successfully creates a user via UserManager.CreateAsync.
- Confirm that ASP.NET Core Identity's password policy correctly rejects weak passwords with descriptive validation errors.
- Document evidence of each test case using Postman response screenshots.

4. Testing Environment

4.1 Tools Used

- Postman — for sending requests and validating responses.
- ASP.NET Core Identity — for user account creation and password validation.

4.2 Endpoint Under Test

POST /api/Auth/register — accepts a username, email, and password, and creates a new user account via UserManager.CreateAsync.

4.3 Base URL Configuration

A Postman environment variable named `{{baseUrl}}` was used in place of a hardcoded host, so the collection can be reused across local, staging, or production environments by updating a single variable.

Example of variable usage in a request URL:

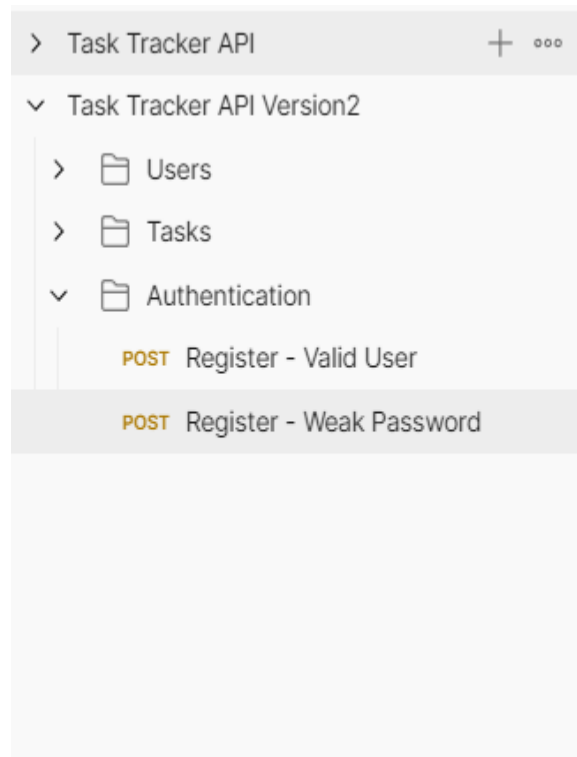
`{{baseUrl}}/api/Auth/register`

5. Postman Collection Structure

All requests were organized inside a single Postman Collection, under an Authentication folder for clarity and maintainability:

Task Tracker API Version 2

- Authentication
 - Register - Valid User
 - Register - Weak Password



Screenshot of the Postman collection sidebar showing the Authentication folder

6. Test Cases — Registration Endpoint

The table below summarizes all test cases executed against the `/api/Auth/register` endpoint, covering a successful registration as well as an expected validation failure (weak password).

Test ID	Method	Endpoint	Expected Result
TC-001	POST	/api/Auth/register (valid data)	200 OK
TC-002	POST	/api/Auth/register (weak password)	400 Bad Request

6.1 TC-001 — Valid Registration

Request Name	Register - Valid User
Method	POST
Endpoint	/api/Auth/register
Body Type	JSON
Input	Valid username, valid email, and a strong password
Expected Result	200 OK
Actual Result	200 OK
Status	✓ Pass

A POST request was sent to the registration endpoint with a valid username, email address, and a strong password. The API returned a 200 OK response, confirming that the user was successfully created using `userManager.CreateAsync`.

Task Tracker API Version2 / Authentication / Register - Valid User

POST `{{baseUrl}}/api/Auth/register` Send

Docs Params Authorization Headers (9) Body Scripts Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify

```

1 {
2   "name": "Test User",
3   "email": "testuser1@example.com",
4   "password": "StrongPassword123!"
5 }

```

Body Cookies Headers (4) Test Results (2/2) 200 OK • 4.75 s • 290 B Save Response

JSON Preview Visualize

```

1 {
2   "message": "User registered successfully.",
3   "userId": "6ee135d1-4ac7-4783-94c4-a65766715654",
4   "name": "Test User",
5   "email": "testuser1@example.com"
6 }

```

Task Tracker API Version2 / Authentication / Register - Valid User

POST `{{baseUrl}}/api/Auth/register` Send

Docs Params Authorization Headers (9) Body Scripts Settings Cookies

Pre-request Post-response

```

1 pm.test("Status code is 200", function () {
2   pm.response.to.have.status(200);
3 });
4
5 pm.test("Registration was successful", function () {
6   const response = pm.response.json();
7   pm.expect(response.message).to.eql("User registered successfully.");
8 });

```

JSON Preview Visualize

```

1 {
2   "message": "User registered successfully.",
3   "userId": "6ee135d1-4ac7-4783-94c4-a65766715654",
4   "name": "Test User",
5   "email": "testuser1@example.com"
6 }

```

Screenshots of the Postman request/response for "Register - Valid User" showing the request body and the 200 OK response

6.2 TC-002 — Weak Password Validation

Request Name

Register - Weak Password

Method	POST
Endpoint	/api/Auth/register
Body Type	JSON
Input	Valid username/email, intentionally weak password
Expected Result	400 Bad Request with validation errors
Actual Result	400 Bad Request with validation errors
Status	✓ Pass

A second POST request was sent using the same endpoint, this time with a deliberately weak password. The API correctly rejected the request with a 400 Bad Request response, returning validation error messages generated by ASP.NET Core Identity's password policy rules.

Task Tracker API Version2 / Authentication / Register - Weak Password

POST `{{baseUrl}}/api/Auth/register` Send

Docs Params Authorization Headers (9) Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

```

1 {
2   "name": "Weak User",
3   "email": "weakuser@example.com",
4   "password": "123"
5 }

```

Body Cookies Headers (4) Test Results (2/2) 400 Bad Request • 224 ms • 569 B • Save Response

{ } JSON Preview Debug with AI

```

1 [
2   {
3     "code": "PasswordTooShort",
4     "description": "Passwords must be at least 6 characters."
5   },
6   {
7     "code": "PasswordRequiresNonAlphanumeric",
8     "description": "Passwords must have at least one non alphanumeric character."
9   },
10  {
11    "code": "PasswordRequiresLower",
12    "description": "Passwords must have at least one lowercase ('a'-'z')."
13  },
14  {
15    "code": "PasswordRequiresUpper",
16    "description": "Passwords must have at least one uppercase ('A'-'Z')."
17  }
18 ]

```

Activate Windows

The screenshot shows a Postman interface for a POST request to `{{baseUrl}}/api/Auth/register`. The request body is a JSON object. The response is a 400 Bad Request with a JSON array of validation errors.

Request Body:

```
1 pm.test("Status code is 400", function () {
2   pm.response.to.have.status(400);
3 });
4
5 pm.test("Weak password is rejected", function () {
6   pm.expect(pm.response.code).to.eql(400);
7 });
```

Response Body (400 Bad Request):

```
1 [
2   {
3     "code": "PasswordTooShort",
4     "description": "Passwords must be at least 6 characters."
5   },
6   {
7     "code": "PasswordRequiresNonAlphanumeric",
8     "description": "Passwords must have at least one non alphanumeric character."
9   },
10  {
11    "code": "PasswordRequiresLower",
12    "description": "Passwords must have at least one lowercase ('a'-'z')."
13  },
14  {
15    "code": "PasswordRequiresUpper",
16    "description": "Passwords must have at least one uppercase ('A'-'Z')."
17  }
18 ]
```

Screenshots of the Postman request/response for "Register - Weak Password" showing the request body and the 400 Bad Request validation errors

7. Test Summary

A total of 2 test cases were executed against the registration endpoint, covering both a success scenario and an expected validation failure.

Total Test Cases	2
Success Scenarios (2xx)	1
Error Scenarios (4xx)	1

8. Conclusion

Both planned test cases were executed successfully using Postman, with results matching the expected HTTP status codes. The registration endpoint correctly handles a valid request — returning 200 OK and creating the user via `userManager.CreateAsync` — and correctly rejects a weak password with a 400 Bad Request response and descriptive validation errors from ASP.NET Core Identity.

This confirms that the identity validation pipeline and registration endpoint meet the expected functional requirements for user creation and password validation. Screenshot evidence for each test case is provided in the sections above and should be inserted at the indicated placeholders.